

Vertex Objectives

Version: 1.0

Status: Draft

1. Introdução

O Vertex é uma plataforma de desenvolvimento e microsistema para smartwatches baseados em Android puro, projetada para oferecer uma arquitetura moderna, modular e previsível.

O projeto nasceu da necessidade de superar limitações encontradas no desenvolvimento tradicional de aplicações para smartwatches, oferecendo uma infraestrutura capaz de evoluir continuamente sem comprometer estabilidade, desempenho ou simplicidade.

O Vertex não pretende substituir o Android. Ele atua como uma camada de plataforma construída sobre ele, fornecendo serviços, APIs e uma arquitetura consistente para aplicações desenvolvidas como plugins.

2. Missão

Criar a melhor plataforma de desenvolvimento para smartwatches Android puros, priorizando modularidade, estabilidade, desempenho previsível e uma experiência de desenvolvimento simples e consistente.

3. Visão

Construir uma plataforma capaz de evoluir continuamente ao longo dos anos, mantendo compatibilidade, alta qualidade de engenharia e facilidade de manutenção, tornando-se uma referência para o desenvolvimento de aplicações em smartwatches Android.

4. Público-alvo

O Vertex é destinado principalmente a:

- proprietários de smartwatches que executam Android puro;
- desenvolvedores interessados em criar aplicações para esses dispositivos;
- pessoas que desejam uma plataforma extensível, estável e altamente modular;
- projetos que necessitam de controle refinado sobre desempenho, recursos e arquitetura.

5. Objetivos

5.1 Plataforma

Fornecer uma infraestrutura completa para o desenvolvimento de aplicações em smartwatches Android.

5.2 Modularidade

Toda funcionalidade deve ser modular sempre que tecnicamente possível.

5.3 Evolução contínua

Permitir que o sistema evolua sem exigir reinstalação completa da plataforma.

5.4 Estabilidade

Falhas isoladas nunca devem comprometer todo o sistema.

5.5 Desempenho previsível

O comportamento do sistema deve permanecer consistente quanto ao uso de CPU, memória, armazenamento e bateria.

5.6 Ergonomia

Toda a plataforma deve ser projetada considerando dispositivos de tela redonda como prioridade.

5.7 Simplicidade

As APIs devem reduzir a complexidade do desenvolvimento, escondendo detalhes internos sempre que possível.

5.8 Escalabilidade

A arquitetura deve suportar crescimento sem necessidade de grandes reestruturações.

6. Princípios Fundamentais

O desenvolvimento do Vertex seguirá permanentemente os seguintes princípios.

Arquitetura modular

Cada componente deve possuir responsabilidades bem definidas.

Separação de responsabilidades

Cada módulo deve possuir uma única responsabilidade claramente identificável.

APIs estáveis

Mudanças devem preservar compatibilidade sempre que possível.

Performance antes de complexidade

Novas funcionalidades nunca devem degradar significativamente o desempenho.

Segurança por padrão

Toda comunicação e acesso a recursos devem ser controlados.

Observabilidade

Todos os componentes devem fornecer informações suficientes para diagnóstico e monitoramento.

Evolução sem reescrita

A arquitetura deve permitir crescimento gradual, evitando reconstruções completas do sistema.

Aprender com os erros da indústria

O Vertex buscará continuamente estudar plataformas, frameworks e sistemas existentes para identificar problemas recorrentes, compreender suas causas e evitar repetir decisões arquiteturais que já demonstraram gerar limitações, complexidade excessiva ou baixa manutenibilidade.

Sempre que possível, decisões importantes deverão ser fundamentadas tanto em boas práticas consolidadas quanto nas lições aprendidas por projetos anteriores.

7. O que o Vertex não pretende ser

O Vertex não pretende:

- substituir o Android;
- modificar o kernel do sistema operacional;
- depender de personalizações específicas de fabricantes;
- permitir acesso irrestrito aos recursos do sistema;

- criar APIs duplicadas quando já existir uma solução consistente;
- sacrificar estabilidade em troca de funcionalidades.

8. Valores

O projeto será guiado pelos seguintes valores:

- simplicidade;
- modularidade;
- estabilidade;
- desempenho;
- previsibilidade;
- segurança;
- escalabilidade;
- qualidade de engenharia;
- manutenibilidade;
- documentação de alta qualidade.

9. Metas Técnicas

O Vertex buscará continuamente:

- minimizar consumo de CPU;
- minimizar consumo de memória RAM;
- minimizar consumo de bateria;
- reduzir tempo de inicialização;
- reduzir tempo de atualização;
- reduzir tempo de recuperação após falhas;
- fornecer APIs consistentes;
- manter compatibilidade entre versões;
- facilitar testes automatizados;
- facilitar manutenção a longo prazo.

10. Filosofia do Projeto

O Vertex é construído sobre a ideia de que uma arquitetura sólida é mais importante do que a quantidade de funcionalidades.

Decisões de curto prazo nunca devem comprometer a qualidade da arquitetura de longo prazo.

A plataforma deve permanecer simples de compreender, previsível de utilizar e preparada para evoluir durante muitos anos.

11. Declaração Final

O Vertex existe para fornecer uma base sólida para o desenvolvimento de aplicações em smartwatches Android puros.

Seu objetivo é combinar modularidade, desempenho previsível, estabilidade e simplicidade em uma arquitetura capaz de evoluir continuamente, preservando qualidade, compatibilidade e facilidade de manutenção ao longo do tempo.

Vertex Principles

1. Objetivo

Os Princípios do Vertex definem as regras fundamentais que orientam todas as decisões arquiteturais, técnicas e de desenvolvimento do projeto.

Eles são permanentes e possuem prioridade sobre decisões de implementação.

2. Arquitetura Primeiro

Toda decisão deve preservar a qualidade da arquitetura.

Nenhuma funcionalidade justifica comprometer a organização do sistema.

3. Separação de Responsabilidades

Cada componente deve possuir uma responsabilidade clara e bem definida.

Mudanças em um componente devem causar o menor impacto possível nos demais.

4. Modularidade

Sempre que tecnicamente viável, funcionalidades devem ser implementadas como módulos independentes.

A modularidade deve facilitar evolução, manutenção e reutilização.

5. Performance por Projeto

A performance deve ser considerada desde o início do desenvolvimento.

O sistema deve evitar desperdícios de CPU, memória, armazenamento e bateria.

O objetivo é manter comportamento previsível mesmo sob carga.

6. Simplicidade

A solução mais simples que atenda corretamente aos requisitos deve ser preferida.

Complexidade somente será adicionada quando houver benefícios claros e mensuráveis.

7. Evolução Contínua

A arquitetura deve permitir evolução sem necessidade de grandes reescritas.

Novos recursos devem integrar-se naturalmente ao sistema existente.

8. Compatibilidade

Sempre que possível, novas versões devem preservar compatibilidade com versões anteriores.

Mudanças incompatíveis devem ser cuidadosamente justificadas.

9. Segurança por Padrão

Todo acesso a recursos deve ser explicitamente autorizado.

Nenhum componente deve receber privilégios além do necessário.

10. Estabilidade

Falhas devem permanecer isoladas.

Um componente defeituoso nunca deve comprometer toda a plataforma.

11. Observabilidade

Todos os componentes devem produzir informações suficientes para diagnóstico, monitoramento e depuração.

Problemas devem ser facilmente identificáveis.

12. APIs Consistentes

As APIs devem seguir padrões uniformes de nomenclatura, comportamento e documentação.

A experiência do desenvolvedor deve ser previsível.

13. Baixo Acoplamento

Os componentes devem conhecer o mínimo possível uns sobre os outros.

Toda comunicação deve ocorrer através de contratos públicos.

14. Alta Coesão

Cada módulo deve concentrar funcionalidades relacionadas entre si.

Responsabilidades não devem ser espalhadas por diferentes partes do sistema.

15. Reutilização

Sempre que possível, soluções já existentes devem ser reutilizadas antes da criação de novas implementações.

Duplicação de código e de conceitos deve ser evitada.

16. Documentação como Parte do Software

Toda funcionalidade importante deve possuir documentação atualizada.

A documentação faz parte do produto.

17. Qualidade Contínua

A qualidade deve ser construída continuamente por meio de testes, revisão de arquitetura, análise de desempenho e inspeções técnicas.

18. Aprender com a Experiência

O Vertex deve estudar continuamente arquiteturas, plataformas e projetos existentes.

Boas ideias devem ser incorporadas.

Problemas já conhecidos pela indústria devem ser evitados sempre que possível.

As decisões arquiteturais devem considerar não apenas o que funcionou em outros projetos, mas também os erros, limitações e dificuldades enfrentados por eles.

19. Pensamento de Longo Prazo

Sempre que houver conflito entre uma solução rápida e uma solução sustentável, deve-se priorizar aquela que ofereça melhores condições de manutenção, evolução e estabilidade ao longo dos anos.

20. O Usuário em Primeiro Lugar

Toda decisão deve buscar melhorar a experiência do usuário final e do desenvolvedor.

A tecnologia existe para servir às pessoas, e não o contrário.

—